Wykład 2

Pracownia Programowania

Instrukcje warunkowe i instrukcje iteracyjne.

Poniższe notatki są na podstawie materiałów Pani Dr. A. Zbrzeżny.

Literatura

Literatura podstawowa

- Stephen Prata
  Język C. Szkoła programowania. Wydanie VI
  Wydawnictwo Helion. Gliwice, kwiecień 2016.
- Stephen G. Kochan
  Język C. Kompedium wiedzy. Wydanie IV.
  Wydawnictwo Helion. Gliwice, listopad 2015.

Literatura uzupełniająca

- Brian W. Kernighan, Dennis M. Ritchie
  Język ANSI C. Programowanie. Wydanie II.
  Wydawnictwo Helion. Gliwice, maj 2010.
- K. N. King (nakład wyczerpany)
  Język C. Nowoczesne programowanie. Wydanie II.
  Wydawnictwo Helion. Gliwice, luty 2011.

Strony internetowe

http://en.cppreference.com/w/c

https://pl.wikipedia.org/wiki/C_(jÄŹzyk_programowania)

https://pl.wikibooks.org/wiki/C

https://pl.wikibooks.org/wiki/C/Biblioteka_standardowa

https://www.tutorialspoint.com/cprogramming/

Przykład (Wypisywanie komunikatu na stadardowe wyjście)

```
#include <stdio.h>

int main(void)
{
    printf("Witajcie w świecie języka C!\n");
    return 0;
}
```

Opis kolejnych fragmentów powyższego programu

- Puste linie są ignorowane.
- Pierwsza niepusta linia programu

```
#include <stdio.h>
```

jest poleceniem preprocesora, powodującym włączenie pliku `stdio.h` przed przejściem do faktycznej kompilacji programu.

- Kolejna niepusta linia

```
// main.h
#include <stdio.h>
#include "point.h"

int main()
{
    struct Point tawerna = makePoint(3, 5);
    showPoint(tawerna);
```

Opis kolejnych fragmentów powyższego programu

- Następna niepusta linia

```
    printf("Witajcie w świecie języka C!\n");
```

  to wywołanie funkcji zadeklarowanej w pliku `stdio.h`. Wywołanie to spowoduje wypisanie na ekranie komunikatu “Hello, World!”. Sekwencja `\n` powoduje przejście do nowej linii.

- Linia

```
    return 0;
```

  kończy funkcję `main()` i zwraca do systemu operacyjnego wartość 0.

- Linia zawierająca tylko nawias `}` kończy ciało funkcji.

Kompilacja i wykonanie programu `hello.c`

- ```
  gcc -Wall hello.c
  ./a.out
  ```
- ```
  gcc -Wall hello.c -o hello
  ./hello
  ```
- ```
  gcc -Wall -Wextra hello.c -o hello
  ./hello
  ```
- ```
  gcc -Wall -Wextra -pedantic hello.c -o hello
  ./hello
  ```

Przykład (Wypisywanie komunikatu na stadardowe wyjście)

```
#include <stdio.h>

int main(void)
{
    printf("Witajcie w świecie języka C!");
    printf("\n");
    return 0;
}
```

Przykład (Komentarze jednoliniowe)

```
#include <stdio.h>

// Prosty program w języku C
// wypisujący na ekran komunikat

int main(void)
{
    printf("Witajcie w świecie języka C!\n");
    return 0;
}
```

Przykład (Komentarze wieloliniowe)

```
#include <stdio.h>

/* Prosty program w języku C
 * wypisujący na ekran komunikat */

int main(void)
{
    printf("Witajcie w świecie języka C!\n");
    // Poniższą instrukcję można pominąć
    return 0;
}
```

Przykład (Wypisywanie wyniku prostych obliczeń)

```
#include <stdio.h>

int main(void)
{
    printf("3 + 5 = %d\n", 3 + 5);
    return 0;
}
```

Przykład (Wypisywanie wyniku prostych obliczeń)

```
#include <stdio.h>

int main(void)
{
    printf("3 + 5 = %d", 3 + 5);
    printf("\n");
    return 0;
}
```

Przykład (Wypisywanie wyniku prostych obliczeń)

```
#include <stdio.h>

int main(void)
{
    int a = 3, b = 5;
    printf("%d + %d = %d", a, b, a + b);
    printf("\n");
    return 0;
}
```

Przykład (Wypisywanie wyniku prostych obliczeń)

```
#include <stdio.h>

int main(void)
{
    int a, b;
    printf("Podaj liczbę całkowitą: ");
    scanf("%d", &a);
    printf("Podaj liczbę całkowitą: ");
    scanf("%d", &b);
    printf("%d + %d = %d", a, b, a + b);
    printf("\n");
    return 0;
}
```

Przykład (Wypisywanie wyniku prostych obliczeń)

```
#include <stdio.h>

int main(void)
{
    double a, b;
    printf("Podaj liczbę zmiennoprzecinkową: ");
    scanf("%lf", &a);
    printf("Podaj liczbę zmiennoprzecinkową: ");
    scanf("%lf", &b);
    printf("%f + %f = %f", a, b, a + b);
    printf("\n");
    return 0;
}
```

Przykład (Wypisywanie wyniku prostych obliczeń)

```
#include <stdio.h>

int main(void)
{
    double a, b, c;
    printf("Podaj liczbę zmiennoprzecinkową: ");
    scanf("%lf", &a);
    printf("Podaj liczbę zmiennoprzecinkową: ");
    scanf("%lf", &b);
    c = a + b; // instrukcja przypisania
    printf("%.2f + %.2f = %.2f", a, b, c);
    printf("\n");
    return 0;
}
```

Przykład (Wypisywanie wyniku prostych obliczeń)

```
#include <stdio.h>

int main(void)
{
    printf("Podaj liczbę zmiennoprzecinkową: ");
    double a;
    scanf("%lf", &a);
    printf("Podaj liczbę zmiennoprzecinkową: ");
    double b;
    scanf("%lf", &b);
    double c = a + b;
    printf("%.2f + %.2f = %.2f\n", a, b, c);
    return 0;
}
```

Przykład (Obliczanie pierwiastka kwadratowego)

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    printf("Podaj liczbę nieujemną: ");
    double a;
    scanf("%lf", &a);
    printf("Pierwiastek z %f = %f", a, sqrt(a));
    printf("\n");
    return 0;
}
```

Priorytety operatorów relacyjnych i logicznych

- Operatory w poniższej tabeli są umieszczone w kolejności malejących priorytetów (im większe pierwszeństwo, tym mniejszy priorytet operatora):

Operator	Opis	Łączność	Pierwszeńswo
`not`	Negacja	prawostronna	2
`<` `<=` `>` `>=`		lewostronna	6
`==` `!=`		lewostronna	7
`and`	Koniunkcja	lewostronna	11
`or`	Alternatywa	lewostronna	12

- Typem wyrażenia logicznego w jezyku C jest `int`.
- Wartością wyrażenia logicznego jest liczba `1`, gdy wyrażenie to jest prawdziwe, a liczba `0` w przeciwnym przypadku.

Przykład (Operatory relacyjne)

```
#include <stdio.h>

int main(void)
{
    printf("1 < 2 =  %d\n", 1 < 2);
    printf("1 <= 2 =  %d\n", 1 <= 2);
    printf("1 > 2 =  %d\n", 1 > 2);
    printf("1 >= 2 =  %d\n", 1 >= 2);
    printf("1 == 2 =  %d\n", 1 == 2);
    printf("1 != 2 =  %d\n", 1 != 2);
    return 0;
}
```

Przykład (Operatory logiczne)

```
#include <stdio.h>

int main(void)
{
    printf("!(1 > 2) = %d\n", !(1 > 2));
    printf("!(1 < 2) = %d\n", !(1 < 2));
    printf("1 < 2 && 3 < 4 = %d\n", 1 < 2 && 3 < 4);
    printf("1 < 2 && 3 > 4 = %d\n", 1 < 2 && 3 > 4);
    printf("1 < 2 || 3 > 4 = %d\n", 1 < 2 || 3 > 4);
    printf("1 > 2 || 3 > 4 = %d\n", 1 > 2 || 3 > 4);
    return 0;
}
```

Przykład (Operatory boolowskie – and, or, not)

```
#include <stdio.h>

int main(void)
{
    printf("5 || 8 = %d\n", 5 || 8);
    printf("5 && 8 = %d\n", 5 && 8);
    printf("(!5) = %d\n", !5);
    printf("!0 = %d\n", !0);
    printf("!5 && 8 = %d\n", !5 && 8);
    printf("8 && 5 < 8 = %d\n", 8 && 5 < 8);
    printf("5 < 8 && 8 = %d\n", 5 < 8 && 8);
    printf("!(5 != 8) = %d\n", !(5 != 8));
    printf("(!5) != 8 = %d\n", (!5) != 8);
    printf("(!5) != 0 = %d\n", (!5) != 0);
}
```

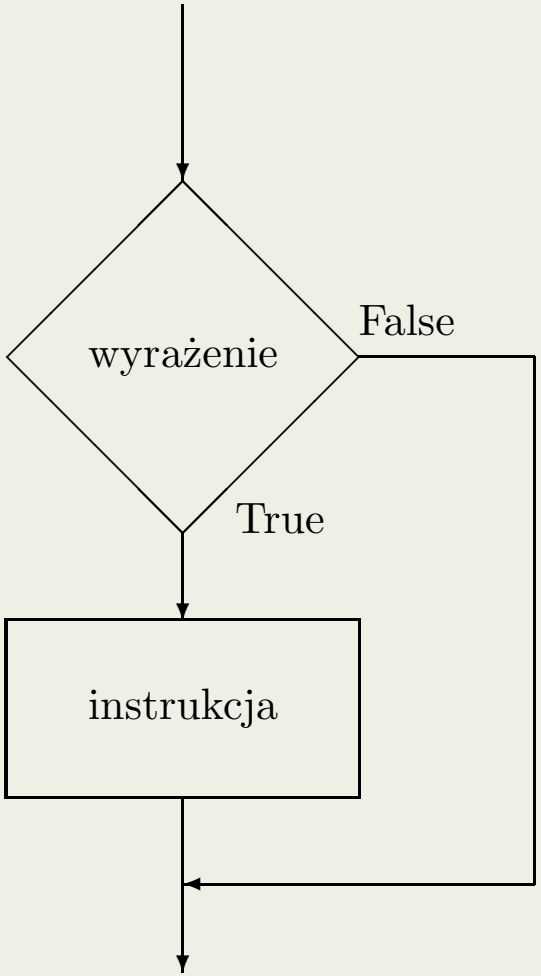
Prosta instrukcja warunkowa

- Składnia prostej instrukcji warunkowej jest następująca:

```
if (wyrażenie) instrukcja
```

- Wykonanie prostej instrukcji warunkowej zaczyna się od obliczenia wartości wyrażenia `wyrażenie`.
- Jeżeli wartość ta jest różna od zera, to wykonywana jest `instrukcja`.
- Jeżeli chcielibyśmy wykonać warunkowo więcej niż jedną instrukcję, to w miejscu tej jednej instrukcji należy zastosować blok instrukcji.
- Blok instrukcji to ciąg instrukcji ujętych w nawiasy klamrowe.
- Blok instrukcji jest traktowany w miejscu, w którym występuje, jako pojedyncza instrukcja.

Schemat blokowy prostej instrukcji warunkowej



Przykład (Obliczanie pierwiastka kwadratowego)

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    printf("Podaj liczbę nieujemną: ");
    double a;
    scanf("%lf", &a);
    if (a < 0) {
        printf("Nie istnieje pierwiastek ");
        printf("rzeczywisty z liczby ujemnej\n");
        return 1;
    }
    printf("Pierwiastek z %f = %f\n", a, sqrt(a));
}
```

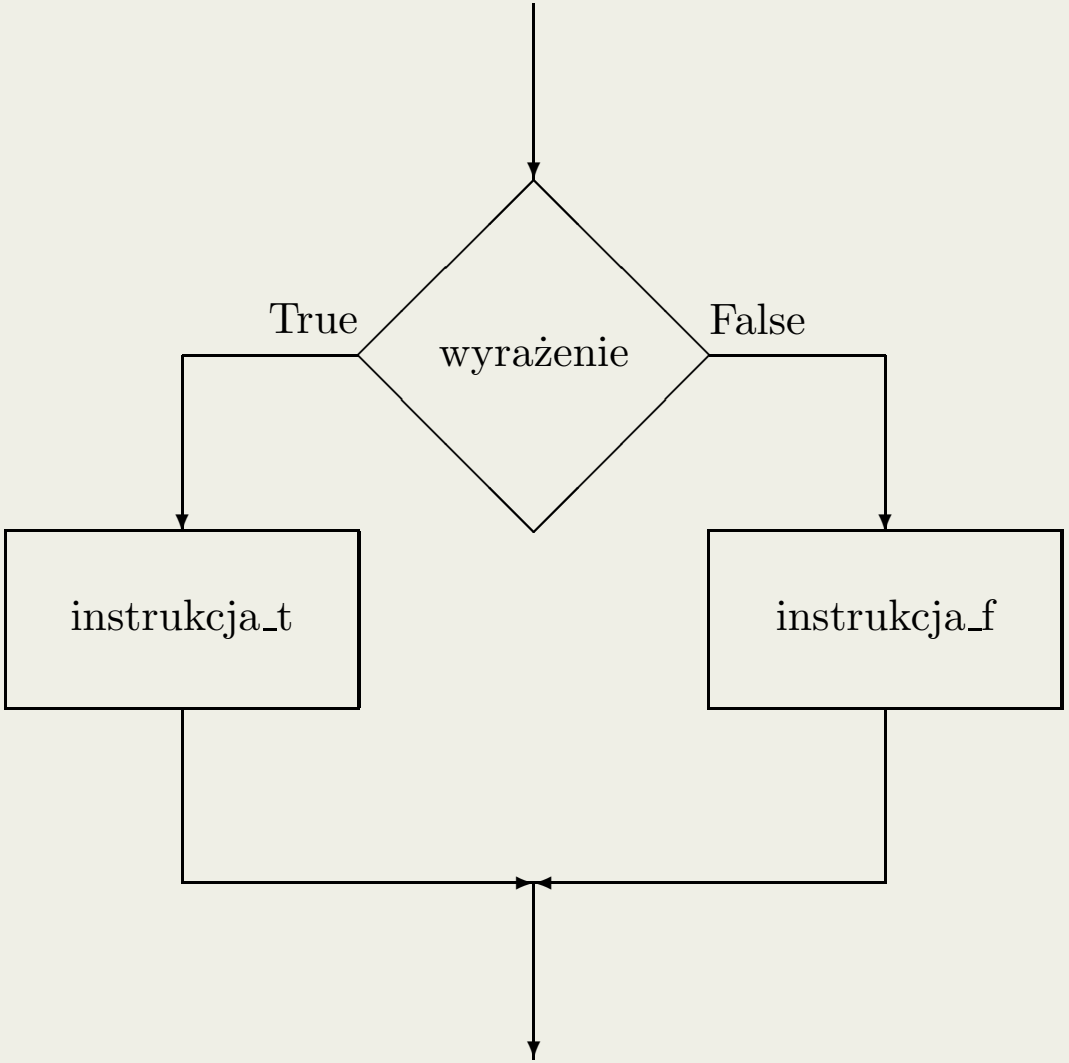
Instrukcja warunkowa

- Składnia instrukcji warunkowej jest następująca:

```
if (wyrażenie) instrukcja_t else instrukcja_f
```

- Wykonanie instrukcji warunkowej zaczyna się od obliczenia wartości boolowskiej wyrażenia `wyrażenie`.
- Jeżeli wartość ta jest różna od zera, to wykonywana jest `instrukcja_t`.
- W przeciwnym przypadku wykonywana jest `instrukcja_f`.
- Zarówno w miejscu występowania `instrukcja_t` jaki i `instrukcja_f`, może wystapić blok instrukcji.

Schemat blokowy instrukcji warunkowej



Przykład (Obliczanie pierwiastka kwadratowego)

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    printf("Podaj liczbę nieujemną: ");
    double a;
    scanf("%lf", &a);
    if (a >= 0) {
        printf("Pierwiastek z %f = %f\n", a, sqrt(a));
    } else {
        printf("Nie istnieje pierwiastek ");
        printf("rzeczywisty z liczby ujemnej\n");
    }
}
```

Instrukcja iteracyjna `while` (`dopóki`)

- Ogólna postać instrukcji iteracyjnej `while` jest następująca:

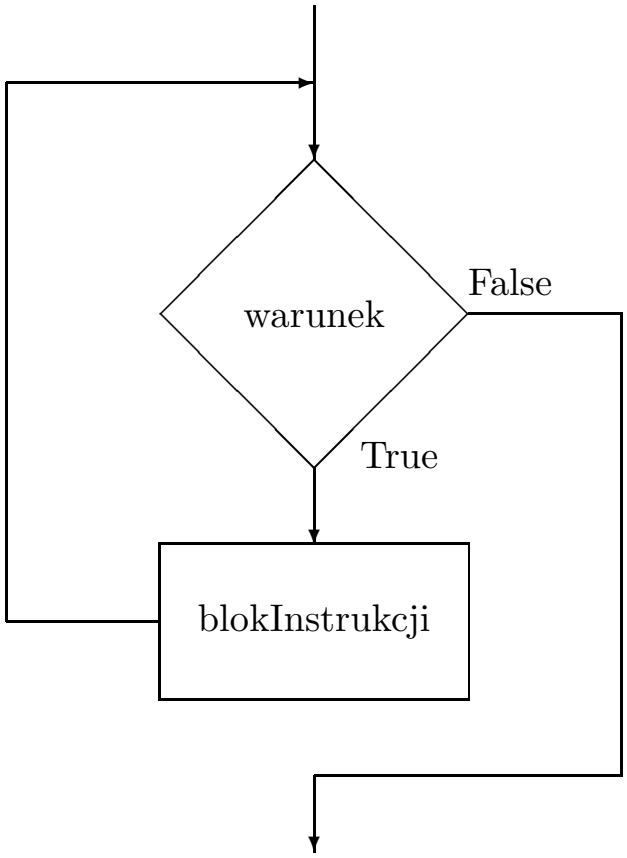
```
while (wyrażenie) instrukcja
```

- Instrukcja iteracyjna `while` jest używana do wykonywania podanej instrukcji tak długo, jak długo wartość wyrażenia `wyrażenie` jest różna od zera.
- Jeżeli chcielibyśmy wykonywać iteracyjnie więcej niż jedną instrukcję, to w miejscu tej jednej instrukcji należy zastosować blok instrukcji.

Semantyka instrukcji iteracyjnej `while`

- Każdorazowo wartościowane jest wyrażenie występujące po słowie kluczowym **`while`**; jeśli jego wartość jest różna od zera, to wykonywana jest `instrukcja`.
- Jeśli za którymś razem (być może już za pierwszym) wartością wyrażenia będzie zero, to nastąpi zakończenie wykonywania instrukcji iteracyjnej.

Schemat blokowy instrukcji iteracyjnej `while`



Przykład (Suma liczb od 1 do n)

```
#include <stdio.h>

int main(void)
{
    printf("Podaj liczbę naturalną: ");
    int n; scanf("%d", &n);
    int s = 0, j = 1;
    while (j <= n) {
        s = s + j;
        j = j + 1;
    }
    printf("Suma liczb od 1 do %d", n);
    printf(" wynosi %d\n", s);
}
```

Przykład (Suma liczb od 1 do n)

```
#include <stdio.h>

int main(void)
{
    printf("Podaj liczbę naturalną: ");
    int n; scanf("%d", &n);
    int s = 0, j = 1;
    while (j <= n) {
        s += j;
        j += 1;
    }
    printf("Suma liczb od 1 do %d", n);
    printf(" wynosi %d\n", s);
}
```

Instrukcja iteracyjna do while

- Ogólna postać instrukcji iteracyjnej do while jest następująca:

```
do instrukcja while(wyrażenie);
```

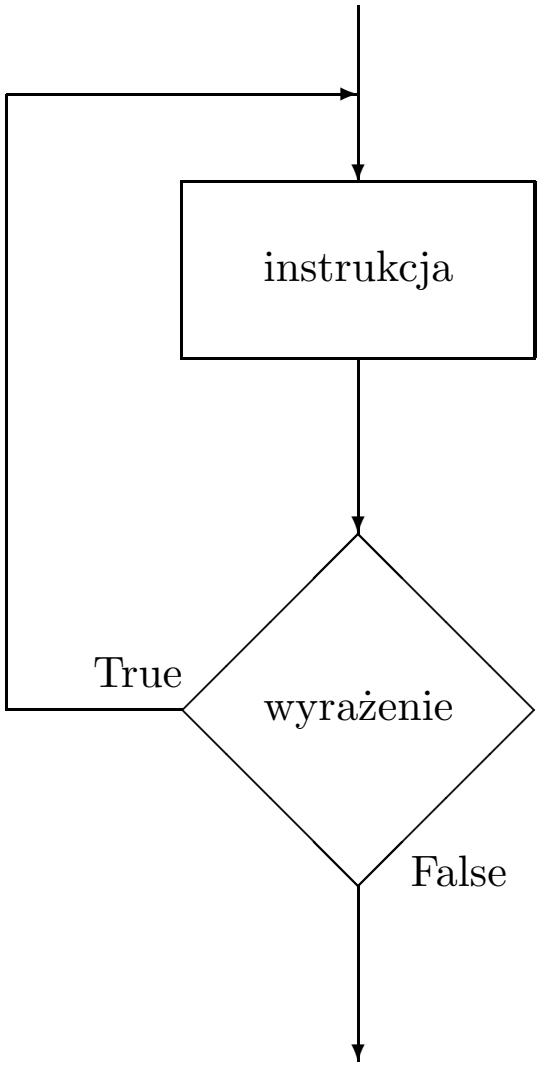
- Preferowany sposób zapisu instrukcji iteracyjnej do while jest następujący:

```
do {
    ciągInstrukcji
} while(wyrażenie);
```

Semantyka instrukcji iteracyjnej `do while`

- Wykonanie instrukcji iteracyjnej `do while` zaczyna się od wykonania instrukcji `instrukcja`.
- Po zakończeniu wykonywaniu instrukcji `instrukcja` wartościowane jest wyrażenie występujące w nawiasach okragłych po słowie kluczowym **`while`** i jeżeli jest ono prawdziwe, następuje powrót do wykonania instrukcji `instrukcja`.
- Jeżeli za którymś razem wartością wyrażenia `wyrażenie` będzie `FALSE`, nastapi zakończenie wykonywania instrukcji iteracyjnej `do while`.

Schemat blokowy instrukcji iteracyjnej `do while`



Przykład (Suma częściowa szeregu harmonicznego)

```
#include <stdio.h>

int main(void)
{
    printf("Podaj liczbę nieujemną: ");
    double a; scanf("%lf", &a);
    double s = 0; int n = 0;
    do {
        n = n + 1;
        s = s + 1.0 / n;
    }
    while (s <= a);
    printf("Liczba składników = %d, suma = %f\n", n, s);
}
```

Priorytety wybranych operatorów

Operator	Opis	Łącz-ność	Prio-rytet
`::`	operator zakresu	L-P	1

`++`
`--`
`()`
	postfiksowa inkrementacja
	postfiksowa dekrementacja
	wywołanie funkcji
		L-P	2

`++`
`--`
`+ -`
`not ~`
(Typ)
`sizeof`
	prefiksowa inkrementacja
	prefiksowa dekrementacja
	unarny plus i minus
	negacja logiczna i negacja bitowa
	rzutowanie
	rozmiar
		P-L	3

`* / %`		L-P	5
`+ -`		L-P	6

Priorytety wybranych operatorów

Operator	Opis	Łącz-ność	Prio-rytet
<< >>	przesunięcia bitowe	L-P	7
< <= > >=		L-P	8
== !=		L-P	9
&	koniunkcja bitowa	L-P	10
^	rozłączna alternatywa bitowa	L-P	11
|	alternatywa bitowa	L-P	12
`and`	koniunkcja logiczna	L-P	13
`or`	alternatywa logiczna	L-P	14

Priorytety wybranych operatorów

Operator	Opis	Łącz-ność	Prio-rytet
? : = += -= *= /= %= <<= >>= &= ^= |=	operator warunkowy operator przypisania operatory przypisania	P-L	15
,	operator przecinkowy	L-P	17

Przykład (Operatory inkrementacji i dekrementacji)

```
#include <cstdio>

int main()
{
   int a = 1;
   printf("a = %d\n", a);
   printf("++a = %d\n", ++a);
   printf("++++a = %d\n", ++++a);
   ++a = 7;
   printf("a = %d\n", a);
   printf("a++ = %d\n", a++);
   printf("a = %d\n", a);
   // printf(++a++); // a++ nie jest l-wartością
   // printf("(++a)++ = %d\n", (++a)++);
   printf("a = %d\n", a);
}
```

Przykład (Operatory inkrementacji i dekrementacji)

```
#include <stdio.h>

int main(void)
{
    int a = 1;
    int b = ++a;
    int c = a++;
    printf("a = %d\nb = %d\nc = %d\n", a, b, c);
}
```

Przykład (Operator przecinkowy)

```
#include <stdio.h>

int main(void)
{
    int a = 1;
    int b = (++a, printf("a = %d\n", a), ++a, 2 * a);
    printf("b = %d\n", (++b, b));
    int c = (++a, ++b);
    printf("Na końcu a = %d\n", a);
    printf("Na końcu b = %d\n", b);
    printf("Na końcu c = %d\n", c);
}
```

Przykład (Trójargumentowy operator warunkowy)

```
#include <stdio.h>

int max(int, int);

int main(void)
{
    printf("max(3, 5) = %d\n", max(3, 5));
    printf("max(5, 3) = %d\n", max(5, 3));
}

int max(int x, int y)
{
    return x > y ? x : y;
}
```

Przykład (Trójargumentowy operator warunkowy)

```
#include <stdio.h>

short sgn(int);

int main(void)
{
    printf("sgn(6) = %d\n", sgn(6));
    printf("sgn(0) = %d\n", sgn(0));
    printf("sgn(-6) = %d\n", sgn(-6));
}

short sgn(int x)
{
    return x < 0 ? -1 : x  > 0 ? 1 : 0;
}
```

Przykład (Trójargumentowy operator warunkowy)

```
#include <stdio.h>

short sgn(int);

int main(void)
{
    printf("sgn(6) = %d\n", sgn(6));
    printf("sgn(0) = %d\n", sgn(0));
    printf("sgn(-6) = %d\n", sgn(-6));
}

short sgn(int x)
{
    return x <= 0 ? x < 0 ? -1 : 0 : 1;
}
```

Typy podstawowe

Typ	Skrócona nazwa	Zakres w modelu LP64
char		-128 ÷ 127
signed char		-128 ÷ 127
unsigned char		0 ÷ 255
_Bool		0, 1
short int	short	-32’767 ÷ 32’767
unsigned short int	unsigned short	0 ÷ 65535
int		-2’147’483’647 ÷ 2’147’483’647
unsigned int	unsigned	0 ÷ 4’294’967’295
long int	long	$-2^{63} \div 2^{63} - 1$
unsigned long		$0 \div 2^{64} - 1$
long long		$-2^{63} \div 2^{63} - 1$
unsigned long long		$0 \div 2^{64} - 1$
float		
double		
long double		

Formanty funkcji printf

Typ	Przykłady stałych	Formanty
char	'a' , '\n'	%c
_Bool	0 , 1	%i , %u
short int	–	%hi , %hx , %ho
unsigned short int	–	%hu , %hx , %ho
int	12 , –97 , 0xFFE0 , 0177	%i , %x , %o
unsigned	12u , 100U , 0XFFu	%u , %x , %o
long int	12L , –2001 , 0xffffL	%li , %lx , %lo
unsigned long	12UL , 100ul , 0xffeeUL	%li , %lx , %lo
long long	0xe5e5e5e5LL , 500ll	%lli , %llx , %llo
unsigned long long	12ull , 0xffeeULL	%llu , %llx , %llo
float	12.34f , 3.1e–5f , 0x1.5P10	%f , %e , %g , %a
double	12.24 , 3.1e–5 , 0x.1p3	%f , %e , %g , %a
long double	12.34l , 3.1e–5l	%Lf , %Le , %Lg

Przykład (Literały całkowite)

```
#include <stdio.h>

int main(void)
{
    printf("%c\n", '0');
    printf("%d\n", 0);
    printf("%d\n", 0U);
    printf("%ld\n", 0L);
    printf("%lu\n", 0UL);
    printf("%lld\n", 0LL);
    printf("%llu\n", 0ULL);
}
```

Przykład (Zapis liczb całkowitych przy wybranej podstawie)

```
#include <stdio.h>

int main(void)
{
    printf("%d %llu\n", 123, 123ULL);
    printf("%d %ld\n", 0x7B, 0x7BL);
    printf("%d %ld\n", 0173, 0173L);

    // Rozszerzenie GCC
    printf("%d %lld\n", 0b1111011, 0b1111011LL);
}
```

Typy zmiennoprzecinkowe

- `float`
- `double`
- `long double`

Przykład (Literały zmiennoprzecinkowe)

- `3.14`
- `3.14F`
- `3.14L`
- `314E-2`
- `0.314E1`
- `3.14E0`

Przykład (Rozmiary typów zmiennoprzecinkowych)

```
#include <stdio.h>

int main(void)
{
    printf("Typ float ma %zu bajtów\n",
        sizeof(float));
    printf("Typ double ma %zu bajtów\n",
        sizeof(double));
    printf("Typ long double ma %zu bajtów\n",
        sizeof(long double));
    return 0;
}
```

Operator sizeof w zastosowaniu do typów

```
#include <stdio.h>

int main(void)
{
    printf("Typ int ma %zu bajty\n", sizeof(int));
    printf("Typ double ma %zu bajtów\n", sizeof(double));
    printf("Typ size_t ma %zu bajtów\n", sizeof(size_t));
    return 0;
}
```

Operator sizeof w zastosowaniu do zmiennych

```
#include <stdio.h>

int main(void)
{
    int a;
    printf("Zmienna typu int ma ");
    printf("%zu bajty\n", sizeof(a));
    double b;
    printf("Zmienna typu double ma ");
    printf("%zu bajtów\n", sizeof(b));
    size_t c;
    printf("Zmienna typu size_t ma ");
    printf("%zu bajtów\n", sizeof(c));
    return 0;
}
```

Wybrane pozostałe funkcje

- `FILE *tmpfile(void)`
  - Tworzy i otwiera tymczasowy plik binarny w trybie aktualizacji (`"r+b"`). W przypadku błędu zwraca `null`.
  - Plik tymczasowy jest automatycznie usuwany.
- `int remove(nazwaPliku)`
  - Usuwa podany plik. W razie błędu zwraca wartość różną od zera.
- `void rewind(wskaznikPliku)`
  - Ustawia na zero wskaźnik położenia we wskazanym pliku.
- `int feof(wskaznikPliku)`
  - Zwraca wartość różną od zera gdy przy ostatnim czytaniu napotkano koniec wskazanego pliku.

Przykłady

- Program `w11-p08.cc` – funkcja `rewind`

Źródła dodatkowych informacji

Opis omawianych funkcji wraz z przykładowymi programami można znaleźć między innymi na stronach:

- `http://pl.wikibooks.org/wiki/C/Biblioteka_standardowa/Indeks_tematyczny`
- `http://en.cppreference.com/w/c/io`
- `http://www.arturpyszczuk.pl/programming-c.html`